

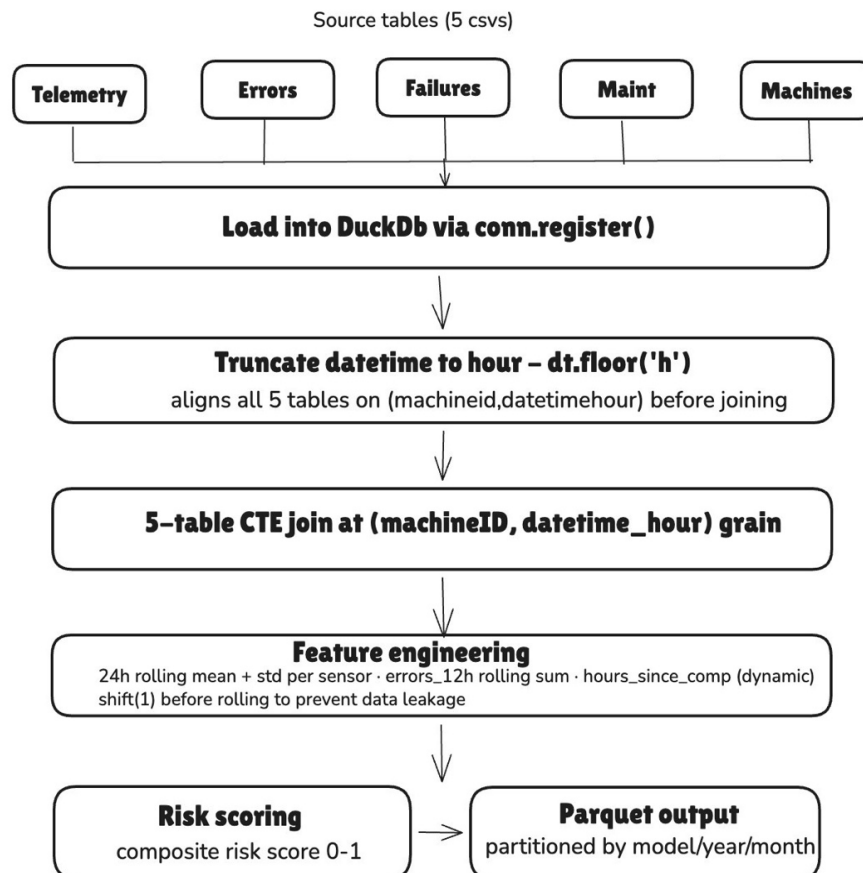
Pipeline Architecture

The pipeline ingests five CSV files into DuckDB using `conn.register()`, which lets SQL queries run directly on pandas dataframes without any server setup.

All datetime columns are floored to the hour using `dt.floor('h')` so every table aligns on the same `(machineID, datetime_hour)` join key before the unified join runs.

Features are engineered on the unified output with 24-hour rolling mean and standard deviation per sensor, a 12-hour error count, and hours since last maintenance per component. A `shift(1)` is applied before every rolling window to prevent the current hour's reading from leaking into its own feature.

The scored output is written to Parquet partitioned by model, year, and month so downstream consumers can filter to just the slice they need without scanning the full dataset.



Part 4 — Production Readiness

Q4.1 — Performance at scale

At 10x volume (roughly 8.76M telemetry rows), the join between telemetry and errors becomes the bottleneck because you're matching millions of hourly rows against tens of thousands of error events. Three Snowflake optimizations I'd apply:

1. **Cluster the telemetry table by (machineID, datetime).**

Snowflake prunes micro-partitions at query time based on the clustering key. A join filter on machineID with a time-range predicate skips all micro-partitions outside the relevant range, which significantly cuts down the data scanned on every pipeline run.

```
ALTER TABLE telemetry CLUSTER BY (machineID, TRUNC(datetime, 'DAY'));
```

2. **Pre-aggregate the errors table before joining with telemetry.**

Aggregating errors down to (machineID, hour) grain before the join reduces the data volume from 39K raw rows to a much smaller set. This avoids the fan-out you'd get from joining raw event-level rows against millions of telemetry rows.

3. **Materialize rolling features as a transient table.**

Snowflake parallelizes window computations across virtual warehouse nodes and writes the result to storage. Subsequent scoring joins then read from the materialized output instead of recomputing the same rolling windows on every run.

Q4.2 — Honest self-assessment: most brittle part

Two things I'd fix if I were hardening this for production:

1. I hardcoded the component list as ['comp1', 'comp2', 'comp3', 'comp4']. That breaks the moment a new component gets added to the maintenance table. The fix is simple — pull the unique values directly from the data so the pipeline adapts automatically without any code changes.

```
components = df_maint['comp'].unique()
```

2. The hours_since_last_maintenance calculation used iterrows() inside a loop over components — essentially $O(n^2)$. For 876K rows that's very slow. The vectorized fix is to flag maintenance events as a binary column and use cumsum() inside a groupby transform. Same result, runs in seconds instead of minutes.

```
for comp in components:
    unified[f'maint_{comp}'] = (unified['last_maint_comp'] ==
                                comp).astype(int)
    unified[f'hours_since_{comp}'] = (
        unified.groupby('machineID')[f'maint_{comp}']
        .transform(lambda x: x.cumsum())
    )
unified.drop(columns=[f'maint_{comp}'], inplace=True)
```